

1. Uvod

Razumevanje izvršavanja programa na nivou asemblerskog jezika podrazumeva praćenje veze između instrukcija, sadržaja registara, memorije i redosleda izvršavanja. Konačan rezultat programa ne pokazuje sva međustanja kroz koja je procesor prošao da bi do njega došao. Zbog toga je za analizu rada programa značajno posmatrati ne samo izlazne vrednosti, već i promene koje nastaju izvršavanjem pojedinačnih instrukcija.

Motivacija za ovaj rad jeste izrada okruženja u kome se kratak asemblerski program može napisati, izvršiti i analizirati uz neposredan uvid u stanje simuliranog procesora. Na primeru petlje koja sabira niz brojeva, korisnik treba da može da prati promenu zbira, vrednost brojača i trenutak u kome uslov grananja prestaje da bude ispunjen. Takav način rada namenjen je povezivanju tekstualnog zapisa programa sa njegovim izvršavanjem, bez potrebe da se za svaki korak ručno vodi evidencija svih promena.

Predmet rada vezan je za RISC-V, porodicu arhitektura skupa instrukcija, odnosno pravila koja određuju instrukcije i stanje procesora vidljivo programu. Specifikacija RISC-V razdvaja osnovni celobrojni skup instrukcija od dodatnih proširenja, pri čemu RV32I označava osnovnu varijantu sa 32-bitnim celobrojnim registrima [1]. U ovom radu razmatra se odabrani podskup instrukcija te varijante, dovoljan za primere aritmetičkih i logičkih operacija, pristupa memoriji i promene toka izvršavanja.

Predmet praktičnog dela rada jeste razvoj veb aplikacije koja objedinjuje editor asemblerskog koda, parser, assembler i simulator izvršavanja. Korisnik unosi program u tekstualni editor, nakon čega aplikacija proverava njegov zapis i prevodi ga u internu reprezentaciju pogodnu za izvršavanje. Ako je program ispravan, prelazi se u režim u kome se može upravljati izvršavanjem i pratiti stanje registara i memorije.

Osnovni cilj je realizacija preglednog alata koji omogućava izvršavanje jedne instrukcije, nastavak izvršavanja do tačke prekida i izvršavanje do završetka programa. Dodatni ciljevi obuhvataju prikaz promena stanja posle korisničke akcije, prijavljivanje grešaka sa vezom ka izvornom kodu i vraćanje simuliranog procesora u početno stanje. Pri tome je potrebno sačuvati vezu između instrukcije koju simulator izvršava i odgovarajuće linije koju korisnik vidi u editoru, kako bi praćenje programa ostalo razumljivo i nakon prevođenja pseudo-instrukcija.

Obim rešenja ograničen je na 15 instrukcija i 5 pseudo-instrukcija. Podržane instrukcije obuhvataju sabiranje i oduzimanje, bitske logičke operacije i pomeranja, učitavanje i upis memorijske reči, uslovno grananje i skokove. Model stanja sadrži 32 registra širine 32 bita, programski brojač i memoriju podataka veličine 4096 bajtova, dok se pseudo-instrukcije u fazi asembliranja prevode u podržane prave instrukcije.

Simulator predstavlja model izvršavanja na nivou instrukcija, a ne model unutrašnje hardverske organizacije procesora. Asemlirani program čuva se kao niz strukturiranih instrukcija, odvojeno od memorije podataka, bez generisanja binarnog mašinskog koda. Ne modeluju se protočna obrada, keš memorija niti trajanje instrukcija u taktovima, pa prikazani rezultati opisuju promene programskog stanja, a ne performanse stvarnog procesora.

Postupak izrade zasnovan je na razdvajanju obrade izvornog koda, izvršavanja i korisničkog interfejsa. Parser izdvaja strukturu linija i operanada, a assembler proverava da li zapis odgovara podržanim instrukcijama i u dva prolaza razrešava labele i priprema program. Model procesora potom izvršava pripremljene instrukcije i menja registre, memoriju i programski brojač, nezavisno od načina na koji se te promene prikazuju korisniku.

Za implementaciju je korišćen TypeScript, dok je korisnički interfejs realizovan veb tehnologijama HTML i CSS i neposrednim radom sa objektnim modelom dokumenta, odnosno DOM-om. Vite se koristi za razvojno okruženje i pripremu aplikacije za isporuku, a Vitest za automatizovane testove. Logika parsera, assemblera i modela procesora ne zavisi od elemenata veb stranice, što omogućava njenu proveru odvojeno od prikaza.

Interakcija sa programom organizovana je kroz režim pisanja i režim izvršavanja. U režimu izvršavanja komanda Step izvršava jednu instrukciju, dok Run nastavlja rad do završetka, tačke prekida ili greške. Prikaz izdvaja sledeću instrukciju, registre u koje je upisivano i memorijske reči koje su menjane, a Reset omogućava ponovno izvršavanje učitanoog programa od početnog stanja.

Prijavljivanje grešaka obuhvata i obradu izvornog teksta i samo izvršavanje. Neispravan zapis instrukcije ili nedefinisana labela sprečavaju pokretanje programa, dok nedozvoljen pristup memoriji ili neispravno odredište skoka prekidaju izvršavanje. Uvedeno je i ograničenje broja instrukcija u jednom pozivu komande Run, čime se ograničava trajanje izvršavanja programa koji ne dolazi do završetka ili tačke prekida.

Provera rešenja zasniva se na automatizovanim testovima komponenti i ručnoj proveru korisničkog interfejsa. Testovi obuhvataju ispravne programe i granične slučajeve, kao što su prekoračenje dozvoljenog opsega konstante, 32-bitno preliivanje, neporavnat pristup memoriji i ponašanje grananja i skokova. Rezultati ovih provera pokazuju kako se implementirane funkcije ponašaju u ispitanim slučajevima. Oni ne predstavljaju formalni dokaz ispravnosti, a na osnovu njih se ne može zaključiti koliko alat pomaže korisnicima u učenju.

Rezultat praktičnog dela jeste integrisana aplikacija u kojoj se program može pratiti od unosa izvornog teksta do promene stanja simuliranog procesora. Doprinos rada je u projektovanju, povezivanju i proveru komponenti koje ostvaruju zadate funkcionalnosti u okviru jasno ograničenog skupa instrukcija. Namenjena je demonstraciji i analizi kraćih asemblerskih programa, dok podrška celokupnom skupu instrukcija i složenijem razvojnom okruženju ostaje izvan obima ovog rešenja.

Rad je organizovan u osam poglavlja. Posle uvoda, drugo poglavlje daje pregled postojećih rešenja i kriterijuma za njihovo poređenje, dok treće objašnjava teorijske osnove potrebne za razumevanje simulatora. Četvrto poglavlje predstavlja korišćene tehnologije, a peto arhitekturu i implementacione detalje sistema.

U šestom poglavlju prikazane su funkcionalnosti aplikacije kroz primer korišćenja i praćenja izvršavanja programa. Sedmo poglavlje opisuje postupak testiranja, rezultate i ograničenja sprovedene provere. Osmo poglavlje donosi zaključak o ostvarenim ciljevima i razmatra moguća unapređenja rešenja.

2. Pregled postojećih rešenja

Za poređenje sa simulatorom razvijenim u ovom radu odabrani su alati RARS i Venus. Oba rešenja namenjena su radu sa RISC-V asemblerskim programima i omogućavaju praćenje njihovog izvršavanja [2] [4]. Pregled je usmeren na mogućnosti značajne za ovaj rad, a ne na iscrpan popis svih funkcija ili rangiranje alata.

2.1. Kriterijumi izbora i poređenja

RARS i Venus izabrani su zato što objedinjuju obradu asemblerskog programa i simulaciju, umesto da zahtevaju fizički procesor za izvršavanje. Njihova dokumentovana podrška za izvršavanje korak po korak i tačke prekida odgovara osnovnim zahtevima razvijenog rešenja [2-4]. Istovremeno predstavljaju dva različita načina korišćenja: RARS se distribuira kao Java aplikacija, dok je Venus dostupan kroz veb pregledač.

Porede se način pokretanja alata, podržani skup instrukcija, kontrole izvršavanja, prikaz registara i memorije, kao i mogućnost uporednog prikaza izvornog i mašinskog koda. Razmatraju se i dodatne mogućnosti, poput vraćanja prethodnog koraka i poziva simuliranog okruženja. Ovi kriterijumi omogućavaju da se funkcionalnosti alata povežu sa konkretnim zadacima: praćenjem petlje, analizom skoka, posmatranjem upisa u memoriju i razumevanjem prevođenja instrukcija.

Pregled je zasnovan na dokumentaciji projekata i dostupnom interfejsu Venus-a. Nisu sprovedena uporedna merenja brzine, sistematska provera usaglašenosti sa specifikacijom niti istraživanje upotrebljivosti sa korisnicima. Na osnovu ovog pregleda mogu se porediti opisane mogućnosti alata, ali se ne može oceniti njihova brzina, potvrditi potpuna ispravnost niti utvrditi koliko pomažu u učenju.

2.2. RARS

RARS, odnosno RISC-V Assembler and Runtime Simulator, predstavlja okruženje za asembliranje i simulaciju RISC-V programa. Distribuira se u obliku izvršne JAR datoteke, za čije pokretanje je prema uputstvu potrebna Java 8 ili novija verzija. Projekat navodi podršku za 32-bitnu i 64-bitnu varijantu arhitekture, više proširenja skupa instrukcija, pozive okruženja i asembliranje programa iz više datoteka [2].

Interaktivno izvršavanje odvija se u prikazu Execute. Komanda Step omogućava praćenje pojedinačnih instrukcija, a Go neprekidno izvršavanje ili nastavak nakon zaustavljanja; tačke prekida postavljaju se uz prikazane instrukcije. Dokumentacija opisuje i podešavanje brzine simulacije, pauziranje, ponovno pokretanje iz početnog stanja i prikaz registara i memorijskih vrednosti u decimalnom ili heksadecimalnom zapisu [3].

Posebno je značajna mogućnost poređenja pseudo-instrukcija sa osnovnim instrukcijama i mašinskim kodom [2]. Ona omogućava razmatranje veze između pogodnijeg zapisa programa i njegovog stvarnog kodiranja, što u razvijenom simulatoru nije predmet prikaza. RARS podržava i korake unazad, ali dokumentacija naglašava da poništavanje promena registara i memorije ne obuhvata konzolni i datotečki ulaz i izlaz [3].

U odnosu na zadati obim ovog rada, RARS obezbeđuje šire razvojno okruženje. To je relevantno za programe koji koriste dodatne instrukcije, više izvornih datoteka ili pozive okruženja, ali nije zahtev za realizaciju ovde izabranih primera. Potreba za Java okruženjem predstavlja razliku u načinu pokretanja, a ne sama po sebi nedostatak funkcionalnosti.

2.3. *Venus*

Venus je simulator RISC-V instrukcija namenjen obrazovanju. U ovom pregledu razmatraju se projekat kvakil/venus i veb aplikacija na koju taj projekat upućuje, bez pripisivanja mogućnosti drugih izvedenih verzija. Prema dokumentaciji, podržan je RV32IM, odnosno osnovni 32-bitni celobrojni skup instrukcija sa proširenjem za množenje i deljenje [1] [4].

Na kartici Editor korisnik piše program, dok na kartici Simulator prati njegovo izvršavanje. Za upravljanje izvršavanjem dostupne su komande Run, Step, Prev i Reset, a stanje se može pratiti u panelima za registre i memoriju. Listing paralelno prikazuje kolone Machine Code, Basic Code i Original Code, čime se omogućava povezivanje izvornog zapisa sa osnovnim instrukcijama i njihovim kodiranjem [5].

Dokumentacija navodi podršku za tačke prekida, vraćanje prethodnog koraka i određene pozive okruženja putem instrukcije ecall, među kojima su ispis i zahtev za dodatnim memorijskim prostorom [4]. Takvi pozivi pružaju dodatne mogućnosti programa unutar simulatora, ali ne treba ih poistovećivati sa simulacijom kompletnog operativnog sistema. Za potrebe ovog poređenja značajni su kao funkcionalnost koja nije uključena u razvijeno rešenje.

Venus je po načinu pristupa bliži aplikaciji iz ovog rada, jer se koristi kroz veb pregledač. Ipak, dostupnost gotove javne veb aplikacije treba razlikovati od lokalnog pokretanja razvijenog projekta, koje u dokumentovanom razvojnom postupku zahteva Node.js, instaliranje zavisnosti i Vite server. Sličnost platforme zato ne znači da su postupci pripreme i obim podržanih programa jednaki.

2.4. *Uporedni pregled i pozicioniranje razvijenog rešenja*

Jedna važna razlika odnosi se na to šta korisnik može da posmatra nakon asembliranja. RARS i Venus prikazuju i mašinski kod, pa se u njima može analizirati veza između tekstualne instrukcije i njenog kodiranja [2] [4]. Razvijeno rešenje zadržava se na internoj reprezentaciji i promeni stanja, zbog čega nije zamena za ove alate kada je cilj vežbe proučavanje bitskih polja instrukcija.

Druga razlika je način ponavljanja analize. Korak unazad u postojećim alatima omogućava povratak na prethodno stanje u okviru podržanih ograničenja [3] [4]. Razvijeni simulator ne čuva istoriju stanja: Reset vraća program na početak, a do željenog mesta ponovo se dolazi pojedinačnim koracima ili tačkom prekida. Ova razlika je važna kada korisnik želi da ponovi samo jednu instrukciju, umesto celog dela programa koji joj prethodi.

Razlike u podržanim instrukcijama i pozivima okruženja utiču i na prenosivost primera. Program koji u postojećem alatu koristi ispis preko ecall ili množenje ne može se bez prilagođavanja izvršiti u razvijenom simulatoru. Pri izboru primera zato je potrebno proveriti da li sve upotrebljene instrukcije i oblici zapisa pripadaju podržanom podskupu, a ne osloniti se samo na zajedničku oznaku RISC-V.

Sažetak odabranih mogućnosti prikazan je u Tabeli 1. Podaci za RARS i Venus preuzeti su iz navedenih izvora [2-5], dok kolona razvijenog simulatora opisuje implementaciju ovog rada. Oznaka da neka funkcija nije podržana odnosi se na trenutnu realizaciju, a ne na mogućnost njenog naknadnog dodavanja.

Tabela 1. Poređenje odabranih mogućnosti simulatora

Kriterijum	RARS	Venus	Razvijeni simulator
Okruženje	Java aplikacija	Veb aplikacija	Veb aplikacija
Skup instrukcija	32/64-bitni; više proširenja	RV32IM	15 instrukcija iz RV32I
Praćenje izvršavanja	Step, Go i tačke prekida	Step, Run i tačke prekida	Step, Run i tačke prekida
Registri i memorija	Prikaz stanja	Prikaz stanja	Prikaz upisivanih registara i reči
Mašinski kod	Prikazuje se	Prikazuje se	Ne generiše se
Korak unazad	Podrжан uz ograničenja	Podrжан	Nije podrжан
Pozivi okruženja	Podrжani	Podrжani	Nisu podrжani

Poređenje pokazuje da izvršavanje korak po korak, tačke prekida i prikaz stanja već postoje u posmatranim alatima. Doprinos ovog rada zato nije uvođenje tih mogućnosti kao novih, već njihova samostalna realizacija i povezivanje parsera, assemblera, modela procesora i korisničkog interfejsa. Izostavljanje mašinskog kodiranja i dodatnih instrukcija ograničava vrste programa i pojava koje se mogu prikazati.

Razvijeni simulator usmeren je na praćenje kraćih programa u okviru 15 instrukcija i 5 pseudo-instrukcija, uz izdvajanje registara i memorijskih reči u koje je upisivano. Takav obim omogućava da se u narednim poglavljima detaljno objasni ceo put od izvornog teksta do promene stanja. Jednostavniji prikaz je svesno izabran pri projektovanju. Međutim, bez ispitivanja sa korisnicima ne može se tvrditi da je razumljiviji ili da korisnicima više pomaže u učenju od prikaza u RARS-u i Venus-u.

Reference

- [1] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, izdanje 20260120, Introduction: RISC-V ISA Overview. <https://docs.riscv.org/reference/isa/v20260120/unpriv/intro.html> [pristupljeno 28.09.2026.].
- [2] TheThirdOne/rars. RARS — RISC-V Assembler and Runtime Simulator, opis projekta. <https://github.com/TheThirdOne/rars> [pristupljeno 28.09.2026.].
- [3] TheThirdOne/rars. Interactive Debugging Features, dokumentacija projekta. <https://github.com/TheThirdOne/rars/blob/master/src/help/Debugging.html> [pristupljeno 28.09.2026.].
- [4] kvakil/venus. Venus, opis projekta. <https://github.com/kvakil/venus> [pristupljeno 28.09.2026.].
- [5] Venus. Veb interfejs simulatora. <https://venus.kvakil.me/> [pristupljeno 28.09.2026.].